Day 9- Facilitation Guide

Index

(1.5 hrs) ILT

I. Recap

In the last session we learnt about the Table Aliases, Joins, Cross join.

As a quick recap, let's see the key points:

Table Aliases: Table aliases are shorthand names assigned to tables or columns in SQL queries to enhance readability and simplify code.

Joins: Joins are SQL operations that combine rows from different tables based on related columns, creating a unified result set.

INNER JOIN: Returns records that have matching values in both tables.row from the first table with every row from the second.

In this session we will learn about the Inner join,Outer join and Left outer join.

II. Outer Join

MySQL Outer JOINs return all records matching from both tables .
It can detect records having no match in the joined table. It returns NULL values for records of joined tables if no match is found.

Types of OUTER JOINS:

1.Left Join: A left join returns all rows from the left table and the matched rows from the right table. If there is no match, NULL values are returned for columns from the right table.

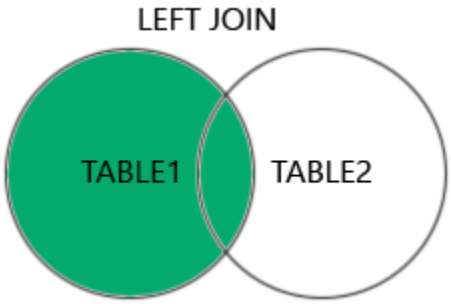
2.Right join: A right join returns all rows from the right table and the matched rows from the left table. If there is no match, NULL values are returned for columns from the left table.

3.Full outer join: A full outer join returns all rows when there is a match in either the left or right table. If there is no match, NULL values are returned for columns from the table with no match.

III. LEFT JOIN

The left join selects data starting from the left table. For each row in the left table, the left join compares with every row in the right table.
If the values in the two rows satisfy the join condition, the left join clause creates a new row whose columns contain all columns of the rows in both tables and includes this row in the result set.

If the values in the two rows are not matched, the left join clause still creates a new row whose columns contain columns of the row in the left table and NULL for columns of the row in the right table.



Here is the basic syntax of a left join clause that joins two tables:
The left join also supports the USING clause if the column used for matching in both tables are the same:

SELECT column_list FROM table_1 LEFT JOIN table_2 USING (column_name);

The following example uses a left join clause to join the Product with the order_details table:

Consider a scenario where you are managing a database for an online retail store. You have two tables: "product" and "order_details."

In this scenario, the SQL query you provided performs a left outer join between the "product" and "order_details" tables based on the common column "product_id." The result set includes details such as product ID, product name, order ID, quantity, order date, and order status. If there is no corresponding order for a product, the columns related to the "order_details" table will display NULL values.

When you execute SQL query:

SELECT product.product_id, product.product_name, order_details.order_id, order_details.quantity,order_details.order_date,order_details.order_status

FROM product
LEFT JOIN order_details ON product.product_id = order_details.product_id;

Output:

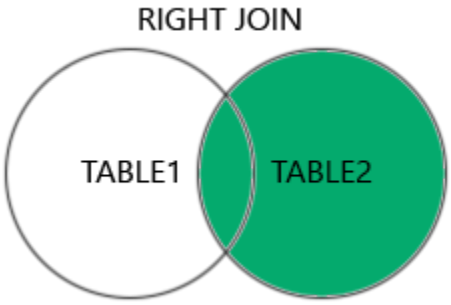
```
mysql> SELECT product.product_id, product.product_name, order_details.order_id, order_details.quantity,order_details.order_date,order_details.order_status
    -> FROM product
    -> LEFT JOIN order_details ON product.product_id = order_details.product_id;
+------------+----------------+----------+----------+---------------------+--------------+
| product_id | product_name   | order_id | quantity | order_date          | order_status |
+------------+----------------+----------+----------+---------------------+--------------+
| P102       | Chair          |        4 |        1 | 2023-11-30 00:00:00 | pending      |
| P102       | Chair          |        5 |        1 | 2023-12-08 00:00:00 | delivered    |
| P102       | Chair          |        8 |        1 | 2023-12-01 00:00:00 | delivered    |
| P103       | Laptop         |        7 |        1 | 2023-12-15 00:00:00 | delivered    |
| P103       | Laptop         |       11 |        1 | 2023-12-21 00:00:00 | pending      |
| P104       | Smartphone     |     NULL |     NULL | NULL                | NULL         |
| P105       | Blender        |        9 |        1 | 2023-11-23 00:00:00 | delivered    |
| P105       | Blender        |       10 |        1 | 2023-12-19 00:00:00 | pending      |
| P106       | Laptop HP      |     NULL |     NULL | NULL                | NULL         |
| P107       | Samsung Galaxy |     NULL |     NULL | NULL                | NULL         |
| P112       | chair          |        3 |        1 | 2023-11-30 00:00:00 | pending      |
+------------+----------------+----------+----------+---------------------+--------------+
11 rows in set (0.00 sec)

mysql>
```

This result set displays product information along with details from the "order_details" table, and NULL values are shown where there is no corresponding order for a product.

IV. RIGHT OUTER JOIN

The RIGHT JOIN clause is similar to the left join clause except that the treatment of left and right tables is reversed. The right join starts selecting data from the right table instead of the left table.

The right join clause selects all rows from the right table and matches rows in the left table. If a row from the right table does not have matching rows from the left table, the column of the left table will have NULL in the final result set.



Here is the syntax of the right join:
SELECT column_list FROM table_1 RIGHT JOIN table_2 ON join_condition;

Similar to the left join clause, the right clause also supports the USING syntax:

SELECT column_list FROM table_1 RIGHT JOIN table_2 USING (column_name);

To find rows in the right table that does not have corresponding rows in the left table, we also use a WHERE clause with the IS NULL operator:

SELECT column_list FROM table_1 RIGHT JOIN table_2 USING (column_name)
WHERE column_table_1 IS NULL;

This statement uses the right join to join the Product and Order_details tables:

Consider a scenario where you are managing a database for an online retail store. You have two tables: "product" and "order_details."

In this scenario, the SQL query you provided performs a right outer join between the "product" and "order_details" tables based on the common column "product_id." The result set includes details such as product ID, product name, order ID, and quantity. If there is no corresponding product for an order, the columns related to the "product" table will display NULL values.

When you execute SQL query:

```
SELECT product.product_id, product.product_name, order_details.order_id,
order_details.quantity
FROM product
RIGHT JOIN order_details ON product.product_id = order_details.product_id;
```

Output:

```
mysql> SELECT product.product_id, product.product_name, order_details.order_id, order_details.quantity
    -> FROM product
    -> RIGHT JOIN order_details ON product.product_id = order_details.product_id;
+------------+--------------+----------+----------+
| product_id | product_name | order_id | quantity |
+------------+--------------+----------+----------+
| P112       | chair        |        3 |        1 |
| P102       | Chair        |        4 |        1 |
| P102       | Chair        |        5 |        1 |
| P103       | Laptop       |        7 |        1 |
| P102       | Chair        |        8 |        1 |
| P105       | Blender      |        9 |        1 |
| P105       | Blender      |       10 |        1 |
| P103       | Laptop       |       11 |        1 |
+------------+--------------+----------+----------+
8 rows in set (0.03 sec)
```

This result set displays product information along with details from the "order_details" table.

Exercise:

Use ChatGPT to explore on INNER JOIN & OUTER JOIN in more deeper:

Put the below problem statement in the message box and see what ChatGPT says.

I want to learn about the difference between INNER JOIN & OUTER JOIN with a real life example.